Sistemas Operacionais — Atividade 2 (AV2)

Memória, armazenamento, sincronização e desempenho de aplicações de IA generativa

Informação	Definição
Modalidade	Trabalho em equipe, de 5 a 7 discentes
Unidade	AV2
Lançamento	03/10/2026
Entrega do repositório e relatório	20/10/2026, às 23h59
Apresentação oral	21/10/2026

1. Contexto

- Aplicações de IA generativa e de recuperação aumentada por geração (RAG) mobilizam memória principal, memória virtual, armazenamento, arquivos, índices, embeddings, caches, filas, logs e mecanismos de sincronização. Nesta atividade, cada equipe deverá implementar, executar e analisar um serviço simplificado denominado GenResearch-Servidor.
- O objetivo é relacionar memória, armazenamento, entrada e saída, caches, filas e sincronização a uma carga de trabalho inspirada em IA generativa. O foco não é demonstrar a melhor qualidade textual da resposta, mas explicar como o sistema operacional administra recursos e quais custos surgem durante a execução.

Pergunta norteadora

- Como memória principal, memória virtual, armazenamento, caches, filas compartilhadas e mecanismos de sincronização afetam o desempenho, a estabilidade e a correção de uma aplicação de IA generativa ou de RAG?

2. Objetivos de aprendizagem

- Observar e explicar RAM, memória virtual, swap, paginação e cache de páginas em Linux.
- Comparar leitura convencional de arquivos com leitura por mapeamento de memória (mmap).
- Identificar efeitos de lote, contexto, volume de documentos e concorrência sobre memória e armazenamento.
- Implementar pipeline simplificado de recuperação e geração com filas e cache compartilhados.
- Demonstrar, de forma controlada, condição de corrida, contenção, starvation ou risco de deadlock.
- Aplicar mutexes, semáforos, filas bloqueantes, monitores ou mecanismo equivalente.
- Medir latência, vazão, RAM, memória virtual, faltas de página, E/S de disco e comportamento de cache.
- Produzir evidências reprodutíveis: código, comandos, dados, logs, gráficos e interpretação técnica.

3. Organização geral da atividade

- Trabalho em equipe, com 5 a 7 discentes.
- Utilizar ambiente Linux: nativo, máquina virtual, WSL ou contêiner, devidamente documentado.
- Usar Python ou outra linguagem previamente aprovada. Ollama ou outro runtime local poderá ser utilizado, desde que a escolha seja documentada.
- A aplicação deverá ser observável por meio de processos, threads, memória, armazenamento, filas, cache, logs e ferramentas do sistema operacional.
- Documentos e dados devem ser públicos, sintéticos ou anonimizados. É proibido publicar chaves, senhas, tokens, arquivos .env, dados pessoais, documentos sigilosos ou pesos de modelos.

4. Cenário do problema

- O GenResearch-Servidor deverá receber documentos e consultas, preparar dados, armazenar documentos e vetores, recuperar contexto, produzir respostas e registrar a execução.
- Os componentes poderão competir por RAM, cache de arquivos, memória virtual, armazenamento local, arquivos de log, índices vetoriais, filas de requisições, caches de respostas e buffers de E/S.
- A equipe deverá escolher e justificar um recorte experimental, sem necessidade de implementar todos os componentes de uma aplicação comercial.

5. Parte A — Observação do ambiente

- Registrar distribuição e versão do sistema operacional; kernel; processador e CPUs; RAM e swap; armazenamento e sistema de arquivos; GPU e VRAM, se houver; ambiente de execução; linguagem e bibliotecas; versão do modelo ou ferramenta; commit do repositório e data da execução.

- Comandos sugeridos: uname -a; cat /etc/os-release; lscpu; free -h; cat /proc/meminfo; lsblk; df -h; swapon --show; vmstat 1; iostat -xz 1.
- Explicar a diferença entre MemFree, MemAvailable, cache e swap; por que o cache de páginas não representa necessariamente memória desperdiçada; e como paginação e leitura de arquivos são relevantes para datasets, índices, embeddings e checkpoints.

6. Parte B — Implementação do GenResearch-Servidor

- Ingestão: ler documentos, fragmentá-los e produzir embeddings reais ou vetores simulados.
- Armazenamento: guardar documentos, metadados e vetores em arquivos, SQLite, FAISS, Chroma ou estrutura equivalente; registrar tamanho do corpus e índice.
- Fila de requisições: receber consultas e registrar tamanho, produtores, consumidores, espera e requisições processadas.
- Recuperação: localizar trechos relevantes, montar contexto limitado e registrar quantidade de trechos, bytes ou tokens aproximados.
- Resposta: gerar por modelo local ou simular custo computacional; registrar tempo e resultado resumido.
- Cache compartilhado: armazenar resultados recentes e medir acertos, erros, substituições e economia de tempo.
- Logs: registrar eventos de ingestão, consulta, recuperação, cache, geração, falha e encerramento; proteger escrita concorrente.

7. Parte C — Experimentos comparativos

- Utilizar o mesmo corpus e a mesma aplicação, salvo quando a comparação envolver uma condição experimental explicitamente definida.
- Executar C1, C2, C3 e C4 obrigatoriamente. Executar C5 e C6 quando possível; caso contrário, justificar tecnicamente e apresentar alternativa equivalente.
- Repetir cada configuração ao menos três vezes e registrar comandos, parâmetros, horário, versões, ambiente, dados e resultados brutos.

7.1 Configurações mínimas

- C1 — Leitura convencional: estabelecer linha de base com open(), read() ou biblioteca padrão.
- C2 — Arquivo mapeado: avaliar o mesmo corpus por mmap.
- C3 — Concorrência e fila: utilizar ao menos dois produtores e dois consumidores, ou configuração equivalente aprovada.
- C4 — Cache compartilhado: comparar consultas repetidas e semanticamente distintas, registrando acertos e erros.
- C5 — Pressão de memória: aumentar corpus, contexto ou lote para observar degradação, paginação ou swap.
- C6 — Falha controlada: testar arquivo ausente, índice indisponível, fila saturada, limite de memória ou falha de serviço externo.

7.2 Variáveis experimentais

- Variar pelo menos três fatores: número de trabalhadores concorrentes (1, 2 e 4), tamanho do corpus, lote, contexto, capacidade da fila, tamanho do cache, política de remoção, uso de mmap, persistência ou padrão de leitura.
- Trabalhadores concorrentes são threads, processos ou tarefas assíncronas que retiram requisições de uma fila e executam recuperação, processamento ou geração. A equipe deverá informar qual mecanismo escolheu e justificá-lo conforme a carga seja limitada por CPU, E/S ou mista.
- Quando C5 ou C6 não puderem ser executadas com segurança, a equipe deverá justificar tecnicamente e apresentar alternativa equivalente.

8. Parte D — Sincronização obrigatória

- Demonstrar compartilhamento de recursos e seu tratamento.
- Opção 1 — Produtores e consumidores: buffer limitado e controle equivalente a empty, full e mutex.
- Opção 2 — Cache compartilhado: demonstrar condição de corrida sem proteção e corrigir com mutex, lock de leitura/escrita, monitor ou equivalente.
- Opção 3 — Banco ou índice compartilhado: demonstrar risco de escrita concorrente ou atualização inconsistente e adotar serialização, lock ou fila de escrita.
- Produzir pelo menos uma falha controlada: atualização perdida, log intercalado, duplicação de cache, consumidor bloqueado, fila saturada, espera circular isolada ou starvation.
- Se houver demonstração de deadlock, utilizar processo isolado, timeout, mecanismo de encerramento e explicação da recuperação.

9. Parte E — Memória virtual e armazenamento

- Experimento recomendado: comparar leitura convencional com leitura por mmap.
- Comparar tempo de abertura e leitura, RSS, memória virtual, faltas de página observáveis, primeira e segunda execução, arquivos pequenos e maiores e implicações para datasets, índices, embeddings e checkpoints.

- Alternativa: aumentar corpus, lote ou contexto e observar RSS, VSZ, MemAvailable, faltas de página, swap, latência e sinais de thrashing. Estabelecer limites explícitos e não comprometer equipamentos ou infraestrutura institucional.

10. Métricas mínimas

- Tempo: ingestão, recuperação, resposta, total, média, mediana e p95.
- Memória: RSS, VSZ, MemAvailable, swap, cache e faltas de página quando disponíveis.
- Armazenamento: tamanho do corpus e índice, bytes lidos/escritos e operações por segundo.
- Concorrência: processos, threads, tamanho médio e máximo da fila, espera e utilização de trabalhadores.
- Cache: taxa de acerto, taxa de erro, tamanho, substituições e tempo economizado.
- Correção: itens perdidos, duplicados, corrupção de log, ordem de processamento e falhas observadas.
- Robustez: erros controlados, retentativas, recuperação e comportamento diante de fila cheia ou vazia.

11. Questões de análise

- Como tamanho de corpus, índice, lote ou contexto afetou RAM, paginação e latência?
- Que diferença foi observada entre leitura convencional e mmap?
- O cache reduziu tempo ou operações de recuperação? Qual foi a taxa de acerto?
- A concorrência aumentou vazão? Em que ponto gerou contenção, fila ou degradação?
- Que recurso compartilhado exigiu exclusão mútua ou sincronização de eventos?
- Que condição de corrida, risco de deadlock, starvation ou falha de consistência foi demonstrada?
- Qual mecanismo de sincronização foi adotado e quais trade-offs introduziu?
- Houve evidência de paginação intensa, swap, faltas de página ou pressão de memória?
- Como os resultados se relacionam a RAG, embeddings, checkpoints e caches?
- Quais limitações impedem generalizar os resultados para modelos maiores, múltiplas GPUs ou ambientes distribuídos?

12. Entregáveis

- Relatório técnico em PDF, entre 8 e 12 páginas, excluídos anexos.
- Repositório público no GitHub, seguindo regra de nome definida pelo docente.
- Código executável, README, dependências e instruções de reprodução.
- Dados sintéticos ou instruções para obtenção de dados públicos.
- Logs, tabelas, gráficos e resultados brutos usados na análise.
- Evidências de processos, threads, chamadas de sistema, memória, armazenamento e sincronização.
- Tabela de contribuição individual.
- Apresentação em PDF ou slides.
- Vídeo de até 20 minutos, com participação de todos, e URL registrada no README, no relatório e em VIDEO.md ou video.txt.

13. Estrutura do relatório

- Tabela de contribuição individual; introdução e problema; pergunta norteadora; ambiente; arquitetura; dados, corpus, embeddings, índice e cache; processos, threads, filas e recursos compartilhados; estratégia de sincronização; metodologia; resultados; análise de memória, armazenamento, paginação e E/S; análise de concorrência, cache e correção; relação com Sistemas Operacionais e IA generativa; limitações; conclusão; referências.

14. Apresentação

- Cada equipe terá 10 a 12 minutos de apresentação e 3 a 5 minutos para perguntas.
- Incluir problema e cenário, ambiente, arquitetura, dados e configurações, processos e recursos, experimentos, resultados e gráfico, sincronização e falha controlada, análise de memória e armazenamento, relação com IA generativa e limitações.
- Todos os integrantes deverão participar da apresentação oral e do vídeo.

15. Observações

- Ausência de código executável ou instruções suficientes para reprodução poderá limitar a nota máxima.
- Resultados sem dados, tabelas, gráficos, logs ou evidências suficientes não serão considerados análise experimental completa.
- Uso de código, texto, configuração ou dados de terceiros deverá ser atribuído e documentado.

- A apresentação deverá demonstrar domínio sobre os artefatos entregues.

16. Roteiro de entrega e marcos

- 03/10/2026 — Lançamento da Atividade 2 e confirmação das equipes.
- 20/10/2026, às 23h59 — Entrega do repositório, relatório em PDF, evidências, vídeo e artefatos solicitados.
- 21/10/2026 — Apresentação oral e fechamento da AV2.